

Least Constraining Value (LCV) Heuristic

The **Least Constraining Value (LCV) Heuristic** is used in **Constraint Satisfaction Problems (CSPs)** to improve the efficiency of the search by choosing the **value** for a variable that **leaves the most flexibility** for the remaining unassigned variables. While **variable selection heuristics** like **Minimum Remaining Values (MRV)** and the **Degree Heuristic** help decide **which variable** to assign next, the **LCV heuristic** guides the choice of **which value** to assign to that variable.

Key Points of LCV Heuristic

1. Definition of LCV Heuristic:

- The **Least Constraining Value (LCV)** heuristic selects the value for a variable that **restricts the domains of other unassigned variables the least**.
- In simpler terms, it looks for the value that leaves the **maximum flexibility** for the remaining variables, reducing the chance of encountering dead-ends in the search.

2. Purpose and Need for LCV Heuristic:

The **LCV heuristic** plays a crucial role in:

- **Avoiding dead-ends:** By selecting the value that affects other variables the least, LCV helps avoid situations where no legal values remain for other variables.
- **Reducing backtracking:** By increasing the flexibility of the search, LCV reduces the likelihood of needing to backtrack, as fewer variable domains will be reduced to empty sets.
- **Maximizing flexibility:** The main goal of LCV is to keep as many options open as possible for future assignments, thereby enhancing the chances of finding a solution more efficiently.

3. When to Use LCV:

- LCV is typically used **after selecting a variable** to assign a value to, particularly when there are multiple legal values available for that variable.
- It is most effective when the problem has **dense constraints**, where assigning a value can heavily affect the remaining unassigned variables.
- LCV is often used in combination with **MRV** or **Degree Heuristic** for variable selection.

How the LCV Heuristic Works

Once a variable has been selected (for example, using MRV), the LCV heuristic chooses the **best value** for that variable by evaluating how each possible value affects the remaining variables. The value that leaves the **fewest constraints violated** (or the most options open for the other variables) is chosen.

Steps of LCV Heuristic:

1. **For each legal value** of the selected variable, calculate how assigning that value will affect the domains of the remaining unassigned variables.
2. **Count the number of constraints** that would be violated or the number of legal values that would be pruned from the domains of the remaining variables if that value is assigned.
3. **Choose the value** that **minimizes the impact** on the remaining variables. In other words, select the value that **prunes the fewest legal values** from other variables' domains.

Example of LCV Heuristic in Action (Map Coloring Problem)

Let's continue with the **Map Coloring Problem**, where the goal is to assign colors to regions of a map such that no two adjacent regions have the same color.

- **Available Colors:** {Red, Blue, Green}
- **Regions:** A, B, C, D, E
- **Constraints:** Adjacent regions must have different colors.

Suppose you have selected **Region A** (using the MRV heuristic, for example), and now you need to assign it a color. The legal values for **Region A** are {Red, Blue, Green}.

Now, consider the effects of each color on the neighboring regions **B**, **C**, and **D**.

- **Option 1: Assign Region A = Red:**
 - This reduces the legal values for **Region B** (adjacent to A) by eliminating Red from its domain.
 - It also reduces the legal values for **Region C** and **Region D** by eliminating Red from their domains.
 - Let's say that assigning Red to A reduces **2 legal values** in total from the domains of B, C, and D.
- **Option 2: Assign Region A = Blue:**
 - This reduces the legal values for **Region B** by eliminating Blue from its domain.
 - Similarly, it reduces the legal values for **Region C** and **Region D**.
 - Let's say that assigning Blue to A reduces **1 legal value** in total from the domains of B, C, and D.
- **Option 3: Assign Region A = Green:**
 - This reduces the legal values for **Region B**, **Region C**, and **Region D** by eliminating Green from their domains.
 - Let's say that assigning Green to A reduces **2 legal values** in total from the domains of B, C, and D.

Using the **LCV heuristic**, we would choose **Blue** for **Region A** because it prunes the **fewest legal values** from the domains of the remaining variables (just 1 legal value).

Mathematical Insight into LCV Heuristic

The **LCV heuristic** can be thought of as an optimization problem where we seek to **minimize the number of domain values pruned** from other variables when a particular value is assigned to the current variable.

Mathematical Formulation:

Let:

- V be the current variable that has been selected.
- $D(V)$ be the domain of V (i.e., the set of possible values for V).
- U be the set of unassigned variables (i.e., variables that have not been assigned values yet).

For each value $v \in D(V)$, the **LCV heuristic** calculates the number of values that assigning v would **remove from the domains** of the remaining variables in U .

Let:

- $\text{prune}(v)$ represent the **number of values** that would be pruned from the domains of the variables in U if V were assigned the value v .

The **LCV heuristic** chooses the value v^* such that:

$$v^* = \arg \min_{v \in D(V)} \text{prune}(v)$$

In other words, choose the value that **minimizes** the number of pruned values from the other variables.

Effect on Remaining Variables:

For each value $v \in D(V)$:

- Evaluate how many legal values would be pruned from the domains of the **neighboring variables** of V .
- The **impact** of assigning v is the total number of values removed from the domains of the neighboring variables.

Thus, the goal of the LCV heuristic is to **minimize the constraint propagation** to other variables, allowing them to retain as many legal values as possible.

Example in N-Queens Problem

In the **N-Queens problem**, the goal is to place queens on an $n \times n$ chessboard such that no two queens attack each other.

Consider the 4-Queens problem. You are placing queens row by row, and the available columns for each row are constrained by the positions of the queens in the previous rows.

Suppose you are placing the queen in **Row 3**, and you have already placed queens in **Row 1** and **Row 2**. The available columns for the queen in **Row 3** are:

- **Columns 1, 2, 3, 4**

Now, for each available column, use the LCV heuristic to determine which column will **prune the fewest legal columns** for the queen in **Row 4**.

- **Option 1: Place Queen in Row 3, Column 1:**
 - This placement may prune several columns for the queen in **Row 4** (due to column and diagonal constraints).
 - Let's assume it prunes 2 legal columns for Row 4.
- **Option 2: Place Queen in Row 3, Column 2:**
 - This placement prunes only 1 legal column for Row 4.
- **Option 3: Place Queen in Row 3, Column 3:**
 - This placement prunes 3 legal columns for Row 4.
- **Option 4: Place Queen in Row 3, Column 4:**
 - This placement prunes 2 legal columns for Row 4.

Using the **LCV heuristic**, you would place the queen in **Row 3, Column 2** because it leaves the most options open for placing the queen in **Row 4** (only prunes 1 legal column).

Forward Checking

Forward Checking is a popular **constraint propagation technique** used in **Constraint Satisfaction Problems (CSPs)** to reduce the search space by eliminating values that would lead to conflicts in future variable assignments. It works by maintaining consistency between the variables by checking how the assignment of a value to the current variable affects the remaining unassigned variables, and by pruning their domains early in the search process.

Key Points of Forward Checking

1. Definition of Forward Checking:

- **Forward Checking** is a **lookahead** strategy in CSPs that checks how the assignment of a value to a variable will affect the possible values (domains) of the remaining unassigned variables.
- If assigning a value to a variable results in the removal of all possible values from the domain of any unassigned variable, **Forward Checking** immediately triggers a **backtrack**.
- The idea is to **prevent conflicts** from occurring in the future by propagating the constraints forward in the search tree.

2. Purpose and Need for Forward Checking:

The **Forward Checking** technique is crucial because it:

- **Detects conflicts early**: By pruning the domains of unassigned variables as soon as a value is assigned to a variable, Forward Checking ensures that potential conflicts are caught early in the search process, preventing unnecessary exploration of infeasible branches.
- **Reduces backtracking**: By removing values that would lead to conflicts from the domains of the remaining variables, Forward Checking reduces the likelihood of reaching a dead-end and needing to backtrack.
- **Improves efficiency**: Forward Checking makes the search process more efficient by reducing the number of unnecessary variable assignments and focusing only on valid ones.

How Forward Checking Works

Forward Checking operates by pruning the domains of unassigned variables as soon as a value is assigned to a variable. Here's how it works:

Steps of Forward Checking:

1. **Assign a Value to a Variable:** The algorithm begins by assigning a value to the current variable (selected using a variable selection heuristic like MRV or the Degree Heuristic).
2. **Check Constraints:** After assigning a value to the current variable, Forward Checking checks the **constraints** between this variable and all the **remaining unassigned variables**.
3. **Prune Inconsistent Values:** For each unassigned variable, Forward Checking removes any value from its domain that would violate a constraint with the assigned value of the current variable.
4. **Check for Domain Wipeouts:** If the domain of any unassigned variable becomes **empty** (i.e., there are no values left that can be assigned without violating constraints), a **backtrack** is triggered because it is impossible to assign a value to that variable.
5. **Continue or Backtrack:**
 - If no domains are wiped out, continue assigning values to the remaining variables.
 - If any domain becomes empty, **backtrack** to try a different value for the previously assigned variable.

Example of Forward Checking in Action:

Let's consider a **3-Queens problem**, where we need to place 3 queens on a 3x3 chessboard such that no two queens are in the same row, column, or diagonal.

- **Variables:** X_1, X_2, X_3 , where each variable represents the column position of the queen in rows 1, 2, and 3, respectively.
- **Domains:** Each variable has the domain $D(X_1) = D(X_2) = D(X_3) = \{1, 2, 3\}$, representing the possible column positions for the queens.
- **Constraints:** No two queens can be in the same column or diagonal.

Step 1: Assign a Value to X_1 :

We assign $X_1 = 1$, meaning we place a queen in the first row and first column.

Step 2: Forward Checking:

Now, Forward Checking checks the impact of this assignment on the domains of the remaining variables X_2 and X_3 :

- X_2 cannot be placed in column 1 (same column as X_1).
- X_2 also cannot be placed in column 2 (diagonal conflict with X_1).
- So, $D(X_2)$ is pruned to $\{3\}$.

Similarly, X_3 cannot be placed in column 1 or column 3 (diagonal conflict with X_1), so $D(X_3)$ is pruned to $\{2\}$.

Step 3: Continue Assignment:

Now we assign $X_2 = 3$ and check how this affects X_3 . However, $X_3 = 2$ is still valid since no conflicts exist with the current assignments of $X_1 = 1$ and $X_2 = 3$.

The final solution is $X_1 = 1, X_2 = 3$, and $X_3 = 2$.

Why Use Forward Checking?

Forward Checking is particularly useful in solving **CSPs** with large search spaces or dense constraints. Here's why it is needed:

1. Early Detection of Failures:

Forward Checking detects failures early in the search process. By pruning the domains of the remaining variables after each assignment, it ensures that we don't pursue paths that would eventually lead to a conflict.

2. Pruning the Search Space:

By reducing the domains of the remaining variables, Forward Checking significantly reduces the size of the search space, making the problem easier to solve. This is especially important in problems with large domains or complex constraints.

3. Reducing Backtracking:

Forward Checking reduces the need for backtracking by preventing dead-ends from occurring in the first place. By propagating the constraints forward, it avoids making assignments that would lead to an inconsistent state later on.

4. Improving Search Efficiency:

Forward Checking improves the overall efficiency of the search process by ensuring that only consistent values are considered for each variable. This reduces the number of variable assignments needed to find a solution.

Arc Consistency

Arc Consistency is a form of **constraint propagation** used to ensure that for every variable in a **Constraint Satisfaction Problem (CSP)**, every value in its domain is consistent with the values in the domains of other variables. It is a vital technique for reducing the search space by eliminating values that cannot be part of any solution.

Key Points of Arc Consistency

1. Definition of Arc Consistency:

- In the context of CSPs, a binary constraint between two variables X and Y is represented as an **arc** (X, Y) .
- A CSP is said to be **arc-consistent** if, for every pair of variables X and Y connected by a constraint, for every value $v \in D(X)$, there is **at least one value** $w \in D(Y)$ such that the constraint between X and Y is satisfied.
- If there is no such value w , the value v is **inconsistent** and is removed from $D(X)$.

2. Purpose and Need for Arc Consistency:

The purpose of enforcing **arc consistency** is to:

- **Prune the search space**
- **Detect inconsistencies early**
- **Speed up search**

Arc Consistency Algorithm (AC-3)

The most commonly used algorithm for enforcing arc consistency is **AC-3**. This algorithm ensures that all arcs in a CSP are consistent by iteratively checking each arc and removing inconsistent values.

AC-3 Algorithm Steps:

1. **Initialize the Queue:** Start with a queue containing all arcs in the CSP.
2. **Process Each Arc:**
 - Remove an arc (X_i, X_j) from the queue.
 - For each value $v \in D(X_i)$, check if there is a corresponding value $w \in D(X_j)$ such that the constraint $C_{i,j}(v,w)$ is satisfied.
 - If no such w exists, remove v from $D(X_i)$.
3. **Recheck Related Arcs:** If any value is removed from $D(X_i)$, add all arcs (X_k, X_i) (where X_k is any variable connected to X_i) back into the queue, as these arcs may now have inconsistent values due to the change in $D(X_i)$.
4. **Repeat Until Queue is Empty:** Continue processing arcs until the queue is empty, at which point the CSP is arc-consistent.

Example of Arc Consistency in Action

Let's consider the **Map Coloring Problem**, where the goal is to color the regions of a map such that no two adjacent regions have the same color.

- **Variables:** A, B, C, D (representing regions).
- **Domains:** Each variable has the domain {Red, Blue, Green}.
- **Constraints:** Adjacent regions must have different colors.
 - A is adjacent to B and C.
 - B is adjacent to A and D.
 - C is adjacent to A and D.
 - D is adjacent to B and C.

Step 1: Initialize the Queue

The queue starts with all the arcs:

$\{(A,B), (A,C), (B,A), (B,D), (C,A), (C,D), (D,B), (D,C)\}$

Step 2: Process the Arc (A, B)

We check the arc (A,B). For each value $v \in D(A)$, we ensure there is a value $w \in D(B)$ such that $A \neq B$. All values in $D(A)$ are consistent with the values in $D(B)$ because no constraints are violated at this point.

Step 3: Process the Arc (A, C)

We check the arc (A,C) similarly and find no inconsistencies. The domain remains unchanged.

Step 4: Process the Arc (B, A)

This is a symmetric check to step 2, so the domain remains consistent.

Step 5: Process the Arc (B, D)

Let's assume that $D(B)=\{\text{Red},\text{Blue}\}$ and $D(D) = \{\text{Red},\text{Blue},\text{Green}\}$. The arc (B,D) requires that $B \neq D$. Since $D(B)$ contains Red and Blue, $D(D)$ must contain values other than Red and Blue. As a result, Green is the only valid color for $D(D)$, so $D(D)$ is pruned to $\{\text{Green}\}$.

Step 6: Process the Remaining Arcs

Continue processing the remaining arcs until no more values can be pruned. The result is a set of arc-consistent variables with pruned domains.

Advantages of Arc Consistency

1. **Early Detection of Failures:** Arc consistency can detect situations where no valid assignments are possible early in the search process, leading to faster detection of unsolvable problems.
2. **Reduction in Search Space:** By eliminating values that cannot be part of any solution, arc consistency significantly reduces the size of the search space, making it easier to find a solution.
3. **Works Well with Other Techniques:** Arc consistency is often used in conjunction with **backtracking search** and other constraint propagation techniques like **forward checking** to further prune the search space and improve efficiency.

General-Purpose Heuristics for CSP

Combining Heuristics

In practice, multiple heuristics are often combined to improve the efficiency of solving CSPs. By strategically choosing variables and values based on different criteria, we can significantly reduce the number of backtracks and improve overall performance.

Here's how various heuristics can be combined:

1. **Use MRV to Select the Variable:**
 - **MRV (Minimum Remaining Values)** is used to select the variable that has the fewest legal values left in its domain. This heuristic helps identify the most constrained variable (i.e., the one most likely to cause failure if not assigned early).
 - **Why MRV First:** This reduces the branching factor by focusing on the most critical part of the problem first.
2. **Use the Degree Heuristic as a Tie-Breaker:**
 - If multiple variables have the same MRV (i.e., they all have the same number of remaining values), use the **Degree Heuristic** to break the tie.
 - **Degree Heuristic** chooses the variable that participates in the most constraints with other unassigned variables. By assigning a value to the most constrained variable in terms of relationships, we reduce the complexity of future assignments.
 - **Why Degree Next:** This helps eliminate conflicts between variables early, reducing the search space for future assignments.
3. **Use LCV to Assign the Value:**
 - Once the variable is selected, use the **LCV (Least Constraining Value)** heuristic to choose the value. The LCV heuristic selects the value that leaves the most legal options available for other variables.
 - **Why LCV:** By minimizing the impact on other variables, LCV reduces the likelihood of conflicts later on, increasing the chances of finding a solution without backtracking.

Heuristic Effectiveness

Different heuristics work effectively in various problem settings. Below is an analysis of the effectiveness of these heuristics:

- **MRV:**
 - Works well when the problem involves highly constrained variables that could cause failure if not addressed early.
 - Especially effective in problems where variables have small domains and are highly interdependent, such as **Sudoku** or **Scheduling**.
- **Degree Heuristic:**
 - Best used as a tie-breaker when multiple variables have the same MRV. It helps reduce future conflicts by assigning values to variables that have the most constraints with other unassigned variables.
 - Particularly effective in **Map Coloring** and **Resource Allocation Problems**.
- **LCV:**
 - Reduces the chance of future conflicts by minimizing the impact of each assignment on remaining variables.
 - Useful in problems where assigning one value can drastically reduce the legal options for other variables, such as in **Timetabling** or **Constraint Scheduling**.

Example: Revisiting the Map Coloring Problem

Let's revisit the **Map Coloring Problem**, where the goal is to assign colors to a map's regions so that no two adjacent regions share the same color. We'll demonstrate how combining the three heuristics (MRV, Degree Heuristic, LCV) can improve efficiency:

- **Step 1:** Use **MRV** to select the region that has the fewest legal colors remaining. Suppose region R1R_1R1 is surrounded by three other regions, making it the most constrained. Assign a color to R1R_1R1 first.
- **Step 2:** If multiple regions have the same MRV, use the **Degree Heuristic** to select the region that is connected to the most other unassigned regions. Suppose R2R_2R2 is connected to two other unassigned regions, making it the most constrained by degree.
- **Step 3:** Use **LCV** to select the color for the next region. Choose the color for R2R_2R2 that leaves the most legal options for the remaining unassigned regions.
- **Step 4:** Apply **Forward Checking** after assigning each color. If the domain of any neighboring region becomes empty, backtrack and try a different assignment.
- **Step 5:** Use **Arc Consistency (AC-3)** periodically to prune inconsistent values from the domains of unassigned regions, ensuring that all values in one region's domain are consistent with the values in adjacent regions.

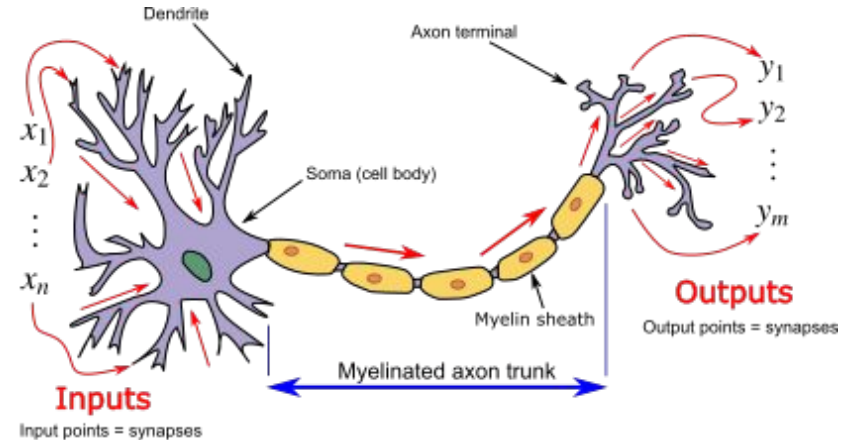
By combining these heuristics and consistency techniques, the problem-solving process becomes more efficient, reducing the search space and the likelihood of backtracking.

Neural Network Basics

Biological Neuron

A typical neuron consists of four major parts:

1. **Soma (Cell Body):**
 - The soma is the central part of the neuron that contains the cell's nucleus.
 - **Function:** The soma integrates incoming signals received from the dendrites.
2. **Dendrites:**
 - Dendrites are branching extensions from the cell body, resembling tree-like structures.
 - **Function:** Dendrites act as the neuron's input receivers.
3. **Axon:**
 - The axon is a long, thin extension from the soma that transmits electrical signals away from the cell body towards other neurons or effector cells (like muscles).
 - **Function:** The axon conducts the electrical signal (known as an **action potential**) from the soma to other neurons. At the end of the axon are terminal branches that connect to other neurons at **synapses**.
4. **Synapses:**
 - The synapse is the small gap or junction between the terminal branches of one neuron and the dendrites or soma of another neuron.
 - **Function:** The synapse is where the neuron communicates with other neurons.

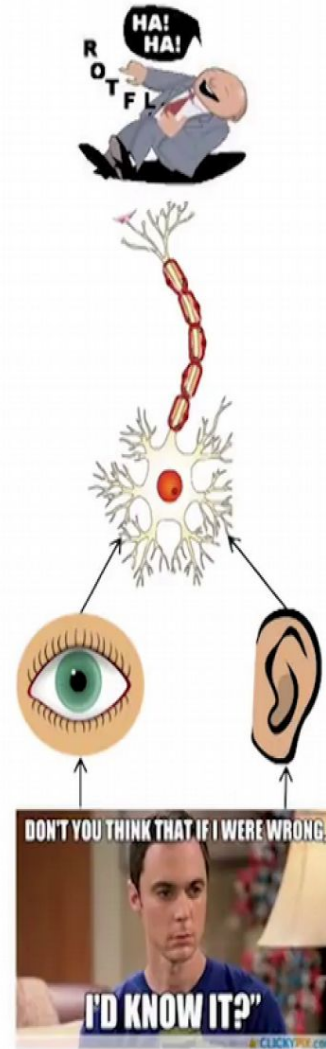


Here we have a cartoonish illustration of how a neuron works.

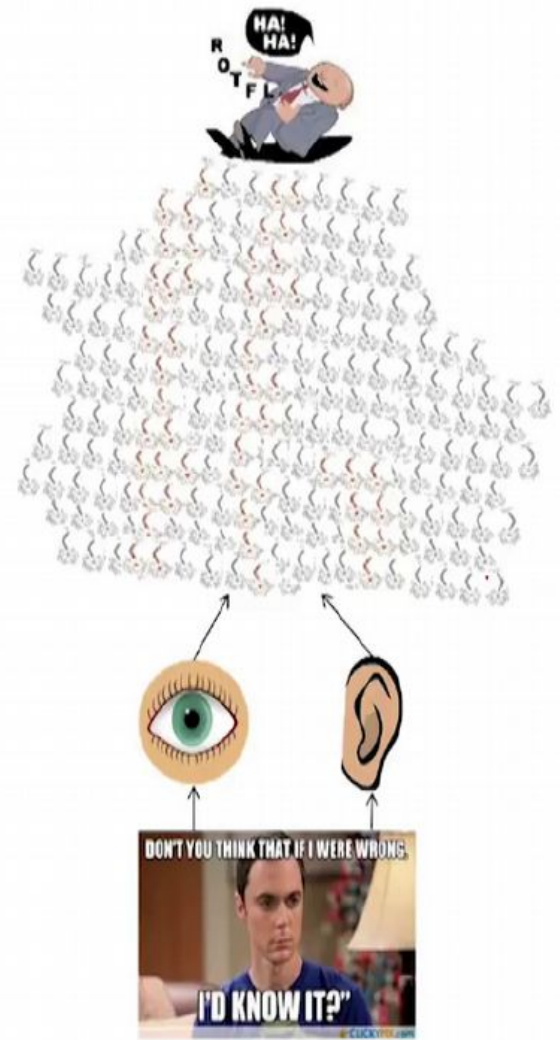
Our sense organs interact with the outside world.

They relay information to the neurons.

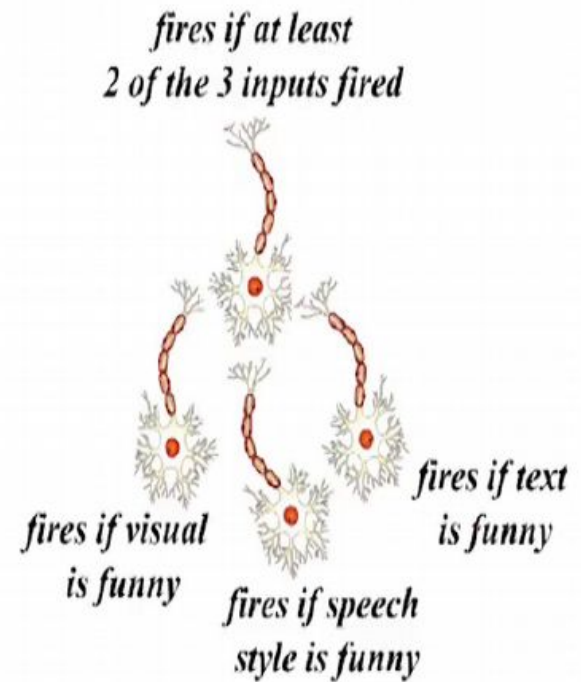
The neurons (may) get activated and produces a response (laughter in this case).



- Ofcourse, in reality, it is not just a single neuron which does all this.
- There is a massively parallel interconnected network of neurons.
- The sense organs relay information to the lowest layer of neurons.
- Some of these neurons may fire (in red) in response to this information and in turn relay information to the other neurons they are connected to.
- These neurons may also fire (again, in red) and the process continues eventually resulting in a response (laughter in this case)
- An average human brain has around 10^{11} (100 billion) neurons!

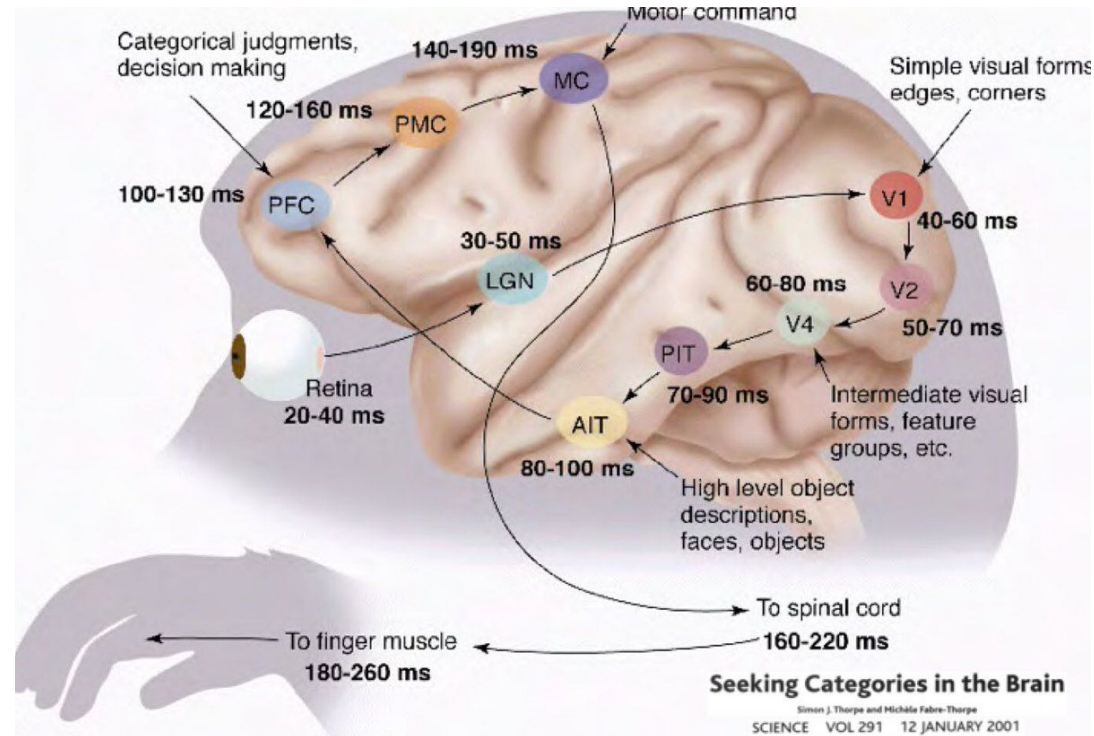


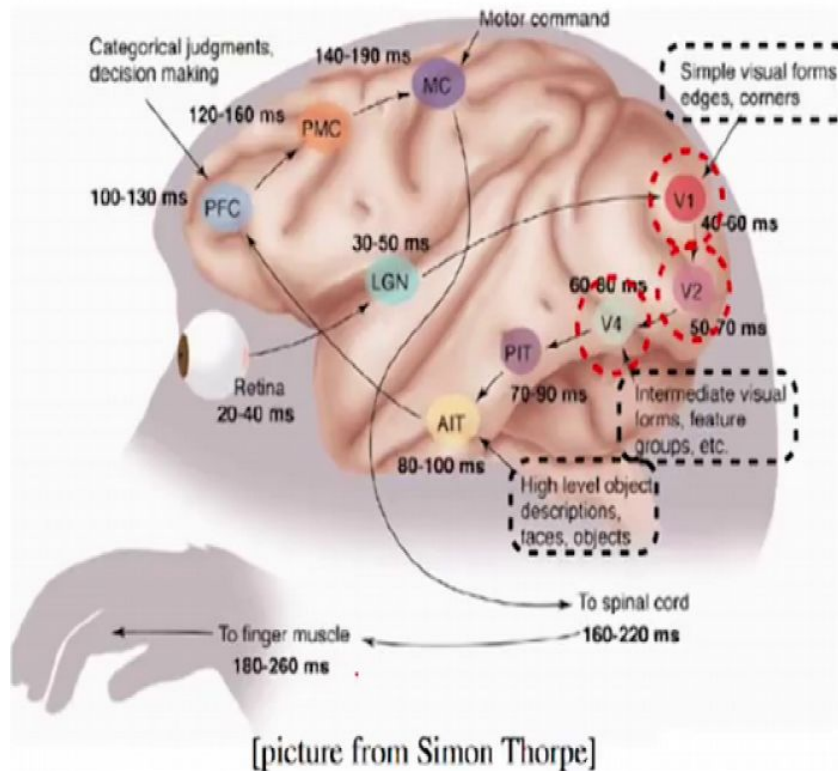
- This massively parallel network also ensures that there is division of work
- Each neuron may perform a certain role or respond to a certain stimulus.



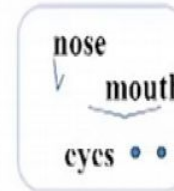
A simplified illustration

- The neurons in the brain are arranged in a hierarchy
- We illustrate this with the help of the visual cortex (part of the brain) which deals with processing visual information.
- Starting from the retina, the information is relayed to several layers (follow the arrows)
- We observe that the layers V1, V2 to AIT form a hierarchy (from identifying simple visual forms to high level objects)





Layer 1: detect edges & corners



Layer 2: form feature groups

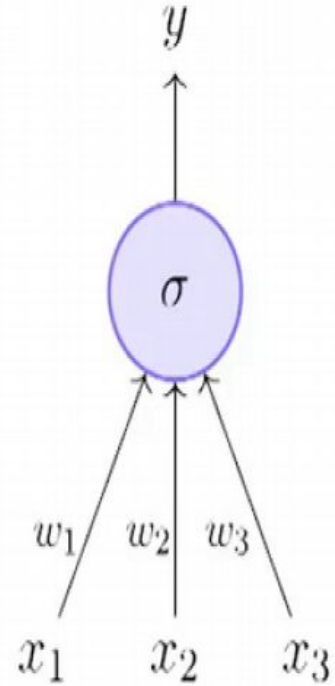


Layer 3: detect high level objects, faces, etc.

Sample illustration of hierarchical processing

Artificial Neuron

The most fundamental unit of a deep neural network is called an artificial neuron.



Artificial Neuron

McCulloch Pitts Neuron

McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)

All the inputs are binary.

MP neuron is divided into two parts: g aggregates the inputs and the function f takes a decision based on this aggregation

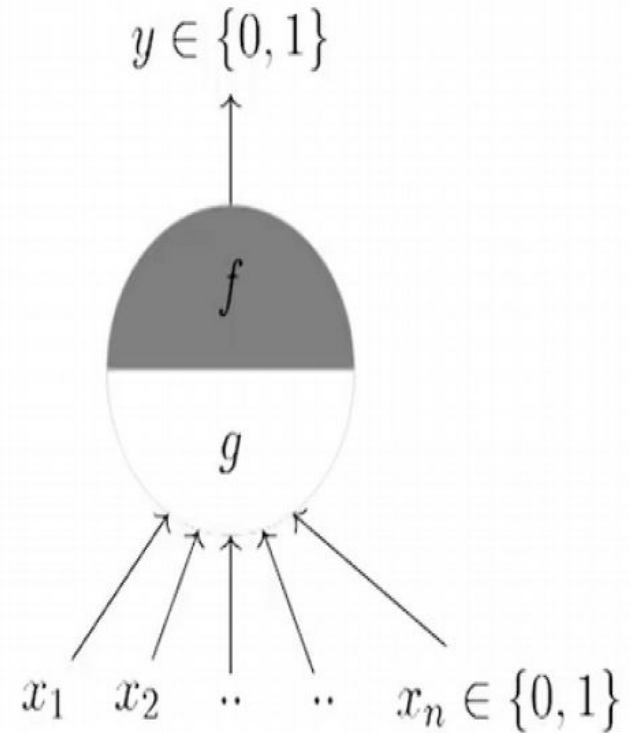
The output is again Boolean.

If it's 0, then neuron does not fire. If it's 1, the neuron fires.

So, suppose I am trying to make a decision whether I should watch a movie or not. So, x_1 could be is the genre of the movie thriller.

Similarly, there could be another variable say x_2 which says is the actor Matt Damon.

So, these are all various such factors that I could take is the director Christopher Nolan, and so on.



$x_1 = isActorDamon$

$x_2 = isGenreThriller$

$x_3 = isDirectorNolan$

The inputs can be excitatory or inhibitory

Inhibitory: inhibitory inputs irrespective of what else is on in your input features. If this input is on, your output is always going to be 0. That means, the neuron is never going to fire.

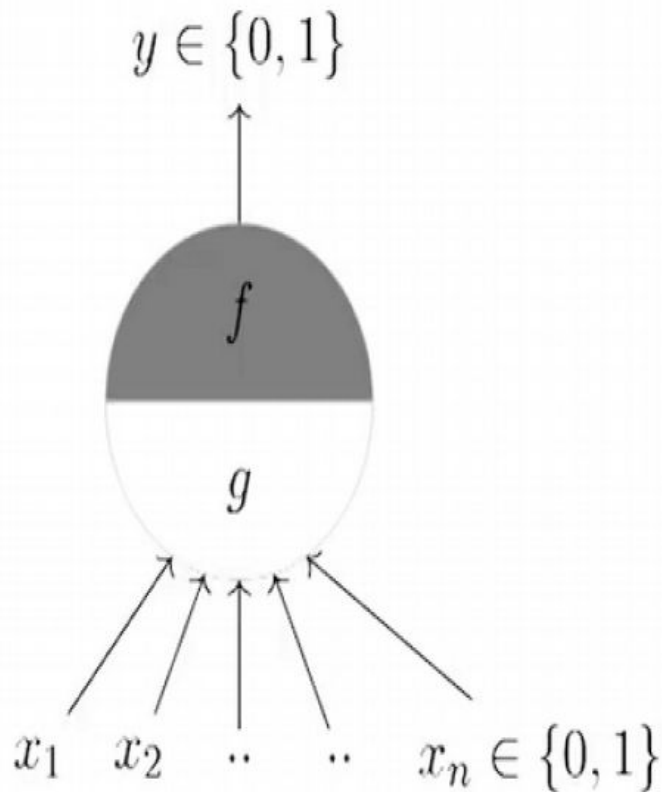
$y = 0$ if any x_i is inhibitory, else

Excitatory: is not something which will cause the neuron to fire on its own, but it combine with all the other inputs that you have seen could cause the neuron to fire

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$\begin{aligned} y = f(g(\mathbf{x})) &= 1 \quad \text{if } g(\mathbf{x}) \geq \theta \\ &= 0 \quad \text{if } g(\mathbf{x}) < \theta \end{aligned}$$

θ is called the thresholding parameter
This is called Thresholding Logic



Perceptron

Frank Rosenblatt, an American psychologist, proposed the classical perceptron model (1958)

A more general computational model than McCulloch-Pitts neurons

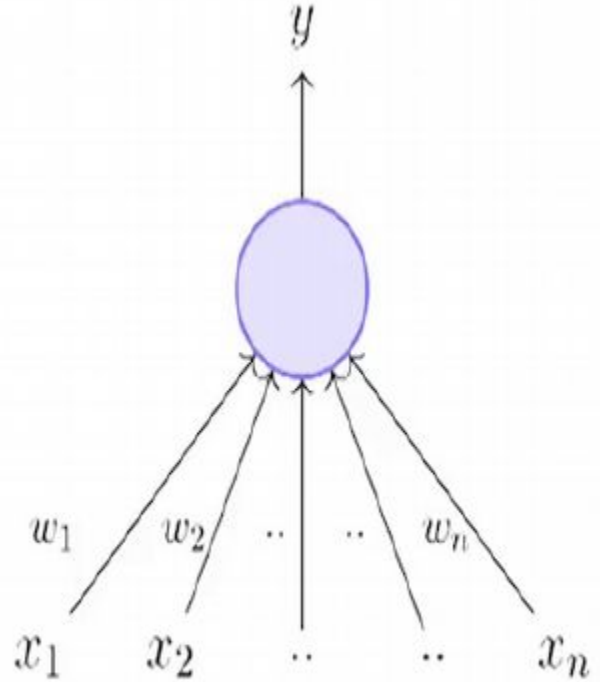
Main differences: Introduction of numerical weights for inputs and a mechanism for learning these weights.

Inputs are no longer limited to boolean values

Refined and carefully analyzed by Minsky and Papert (1969) - their model is referred to as the perceptron model here

It is a type of single-layer neural network.

The perceptron operates as a linear classifier used for binary classification tasks.



- The perceptron takes **multiple inputs** and multiplies each by a corresponding **weight**.
- The inputs are combined into a **weighted sum**, and a **bias** is added.
- This weighted sum is passed through an **activation function** (usually a **step function**).
- The activation function outputs a **1** or **0**, depending on whether the weighted sum crosses a **threshold**.

Inputs: $x_1, x_2, \dots, x_n$.

Weights: $w_1, w_2, \dots, w_n$ applied to each input.

Weighted sum: $z = w_1x_1 + w_2x_2 + \dots + b$, where b is the bias.

Activation function (Step function):

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Output: Binary result — **1** (positive class) or **0** (negative class).

Structure of a Single-Layer Perceptron

The perceptron is a basic unit of a neural network, consisting of the following key components:

a. Inputs ($x_1, x_2, \dots, x_n$)

- The perceptron takes **multiple input features**, which are represented as $x_1, x_2, \dots, x_n$.
- These inputs represent the features of the data to be classified.

Example:

- For a student exam classification task:
 - x_1 : Number of hours studied.
 - x_2 : Number of practice exams taken.

b. Weights ($w_1, w_2, \dots, w_n$)

- Each input has an associated **weight**. Weights determine the importance of each input in making the classification decision.
- These weights are learned during the training process.

Example:

- w_1 : Weight for hours studied.
- w_2 : Weight for the number of practice exams.

c. Bias (b)

- The **bias** term is a constant that is added to the weighted sum of the inputs. The bias allows the perceptron to shift the decision boundary, helping the model adjust its predictions.
- Bias is particularly important when the data points are not centered around the origin.

Example:

- In the student exam classification task, the bias term can help adjust the threshold for determining whether a student passes or fails.

d. Summation of Inputs

- The perceptron takes all input features and multiplies them by their corresponding weights.
- Then, the bias b is added to the weighted sum of the inputs.

The **weighted sum** is calculated as:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Example:

- Suppose the weights and inputs are:
 - $x_1 = 10$ (hours studied).
 - $x_2 = 3$ (practice exams taken).
 - $w_1 = 0.8$ (importance of hours studied).
 - $w_2 = 0.5$ (importance of practice exams).
 - Bias $b = -5$.
- The weighted sum is:
 $z = (0.8 \times 10) + (0.5 \times 3) + (-5) = 8 + 1.5 - 5 = 4.5$

Supervised Learning in the Perceptron

- **Supervised Learning:**
 - The perceptron is trained using **labeled data** where both the input features and their correct output labels are provided. The perceptron learns from these **input-output pairs** to minimize classification errors.
 - Each training example consists of an input vector $x = [x_1, x_2, \dots, x_n]$ and its corresponding correct class label y_{true} , which can be either 0 or 1 in binary classification.
- **Example:**
 - Given input features (e.g., hours studied, number of practice exams taken), the correct label might be "Pass" (1) or "Fail" (0) for each student.

Perceptron Learning Rule

The **learning rule** for the perceptron is based on adjusting the weights and bias whenever the perceptron makes an incorrect prediction. The goal is to reduce the error in classification for future predictions by modifying the weights in proportion to the error.

a. Correct Prediction: No Weight Change

- If the perceptron's predicted label y_{pred} matches the true label y_{true} , no change is made to the weights or bias.

Example:

- If the perceptron predicts **Pass (1)** and the true label is also **Pass (1)**, the perceptron has made the correct prediction, and the weights remain the same.

b. Incorrect Prediction: Update Weights

- If the perceptron's prediction is incorrect ($y_{\text{pred}} \neq y_{\text{true}}$), the weights are updated according to the difference between the true label and the predicted label.

Weight Update Rule:

- For each weight w_i , the update rule is:

$$w_i \leftarrow w_i + \eta(y_{\text{true}} - y_{\text{pred}})x_i$$

Where:

- w_i : The weight associated with the input feature x_i .
- η : The **learning rate**, a small positive constant that controls how much the weights are adjusted in response to the error.
- y_{true} : The correct label (either 0 or 1).
- y_{pred} : The perceptron's predicted label.
- x_i : The value of the input feature associated with weight w_i .

Explanation:

- If the prediction is **incorrect**, the weight is adjusted by an amount proportional to both the learning rate and the input feature. The adjustment moves the weights in the direction that would reduce the error on this particular example.

Example:

- Suppose the perceptron predicts **Fail (0)** for a student, but the true label is **Pass (1)**.
- Input feature $x_1 = 10$ (hours studied) and weight $w_1 = 0.8$.
- The learning rate $\eta = 0.1$
- The weight update for w_1 would be:

$$w_1 \leftarrow w_1 + \eta (y_{\text{true}} - y_{\text{pred}}) \times 1$$

$$w_1 \leftarrow 0.8 + 0.1 (1 - 0) \times 10 = 0.8 + 1 = 1.8$$

- The weight w_1 is increased because the perceptron under-predicted (it predicted 0 when the true label was 1).

Bias Update

Just as the weights are updated when the perceptron makes an incorrect prediction, the **bias** is also updated to shift the decision boundary. The bias update rule follows a similar pattern to the weight update rule.

Bias Update Rule:

$$b \leftarrow b + \eta (y_{\text{true}} - y_{\text{pred}})$$

- b : The bias term.
- η : The learning rate.
- y_{true} : The correct label (either 0 or 1).
- y_{pred} : The perceptron's predicted label.

Explanation:

- If the perceptron under-predicts (predicts 0 when it should predict 1), the bias is **increased** to make it easier for the perceptron to output 1 in the future.
- If the perceptron over-predicts (predicts 1 when it should predict 0), the bias is **decreased** to make it harder for the perceptron to output 1.

Example:

- If the bias $b = -1$ and the perceptron incorrectly predicts **Fail (0)** when the true label is **Pass (1)**, and the learning rate $\eta = 0.1$, the bias is updated as follows:

$$b \leftarrow -1 + 0.1 (1-0) = -1 + 0.1 = -0.9$$

- The bias is increased, making it easier for the perceptron to predict 1 next time.

Perceptron Learning Algorithm

The perceptron learning algorithm iteratively adjusts the weights and bias for each training example. Here's a step-by-step breakdown:

a. Initialize Weights and Bias

- Start by initializing the weights $w_1, w_2, \dots, w_n$ and bias b to small random values.

b. For Each Training Example:

1. Compute the Weighted Sum:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

2. Apply the Activation Function:

- The perceptron outputs either **1** or **0** using the step function:

$$y_{\text{pred}} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

3. Update Weights and Bias if the Prediction is Incorrect:

- If $y_{\text{true}} = y_{\text{pred}}$, do nothing.
- If $y_{\text{true}} \neq y_{\text{pred}}$, update the weights:

$$w_i \leftarrow w_i + \eta(y_{\text{true}} - y_{\text{pred}})x_i$$

- Update the bias:

$$b \leftarrow b + \eta(y_{\text{true}} - y_{\text{pred}})$$

c. Repeat for Multiple Iterations (Epochs)

- The algorithm is repeated for multiple **epochs** (passes over the training dataset) until the perceptron classifies all examples correctly or until a stopping criterion is met (e.g., a set number of epochs or acceptable classification accuracy).

The **convergence theorem** states that the perceptron will eventually find a solution (a set of weights and bias) that perfectly separates the two classes, but only if the data is **linearly separable**.

a. Definition of Convergence

- **Convergence** refers to the perceptron's ability to correctly classify all training examples after a finite number of updates to the weights and bias.
- This occurs when the perceptron has **learned a decision boundary** that perfectly separates the two classes.
- Once the perceptron has converged, the weights and bias no longer change, and the model can make accurate predictions on new, unseen data that is similar to the training data.

b. Linearly Separable Data

- **Linearly separable** means that there exists a straight line (in 2D) or a hyperplane (in higher dimensions) that can perfectly separate the data points into two classes.
- If the data is linearly separable, the perceptron will always find the appropriate weights and bias to correctly classify the data in a **finite number of iterations**.

Example:

- Consider a dataset of points labeled as "A" and "B". If these points can be separated by a straight line, then the perceptron will eventually find this line, and the model will correctly classify every point in the training set.

c. Perceptron Learning Process

- During each iteration of training, the perceptron adjusts its weights and bias based on any misclassified examples.
- With each weight update, the decision boundary moves closer to the correct position.
- If the data is linearly separable, the perceptron will reach a state where no misclassifications occur, and the weights and bias stop changing.

Convergence in Practice:

- **Real-world linear separability:** In practice, perfectly linearly separable data is rare. However, for datasets that are approximately linearly separable, the perceptron will still perform well after training.

Example of Non-linearly Separable Data: The XOR Problem

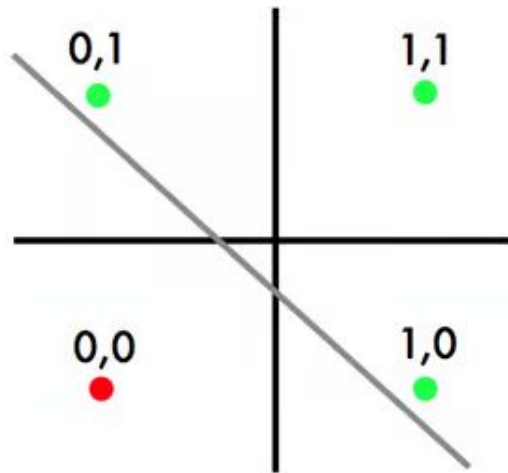
a. What is the XOR Problem?

- The **exclusive OR (XOR)** problem is a classic example of a problem that is **not linearly separable**.
- In the XOR problem, there are four points: two points labeled **1** (positive class) and two points labeled **0** (negative class). The points are arranged in such a way that no straight line can separate the positive and negative points.

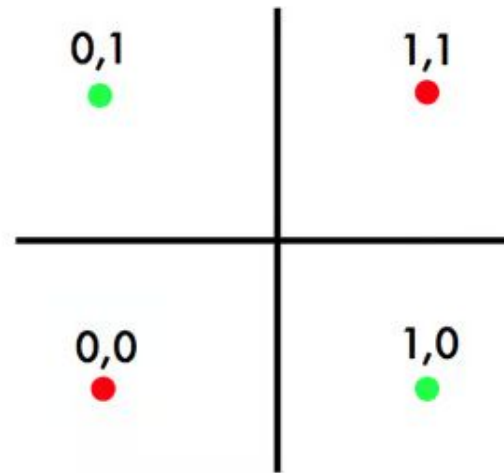
XOR Truth Table:

- $(0,0) \rightarrow 0$ (negative class)
 - $(1,0) \rightarrow 1$ (positive class)
 - $(0,1) \rightarrow 1$ (positive class)
 - $(1,1) \rightarrow 0$ (negative class)
- In the 2D feature space, these points cannot be separated by a straight line because the positive and negative points are arranged diagonally.

The XOR problem



OR



XOR

b. Why Does the Perceptron Fail on XOR?

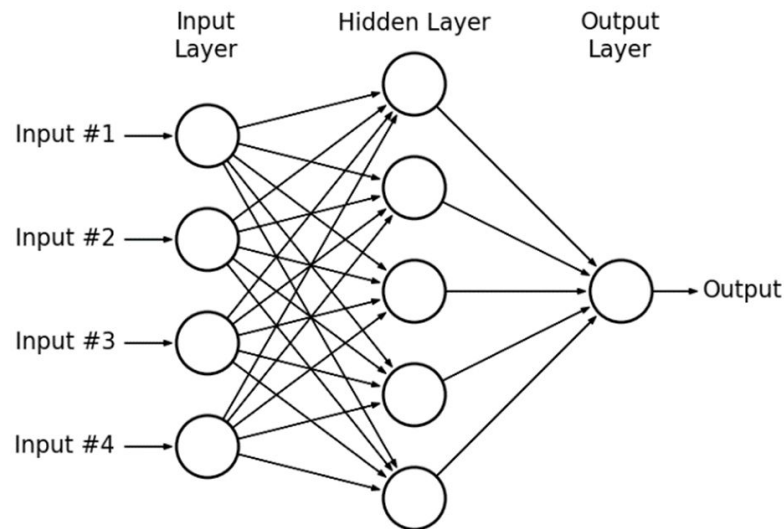
- The perceptron is limited to drawing a **linear boundary** (a straight line) between classes. Since the XOR points cannot be separated by a single line, the perceptron **fails to converge**.
- In this case, the perceptron would never be able to adjust its weights and bias in a way that results in correct classification for all points.

Solution to Non-linear Problems: Multilayer Perceptron (MLP)

- The failure of the perceptron on problems like XOR led to the development of more advanced models, such as the **multilayer perceptron (MLP)**.
- An MLP contains multiple layers of neurons (input, hidden, and output layers), allowing it to model more complex, non-linear decision boundaries.
- The key innovation in the MLP is the use of **non-linear activation functions** (such as the sigmoid or ReLU function) in the hidden layers. These functions enable the MLP to approximate complex functions and solve problems that cannot be solved by a single-layer perceptron.

What is a Multilayer Perceptron (MLP)?

- The **Multilayer Perceptron (MLP)** is a type of **feedforward artificial neural network (ANN)**.
- In an MLP, data flows in one direction—**forward**—from the input layer, through the hidden layer(s), to the output layer, without any feedback loops.
- Unlike a **single-layer perceptron**, which only has an input and output layer, the MLP includes one or more **hidden layers**, enabling it to learn more complex patterns and relationships.



Fully Connected Layers

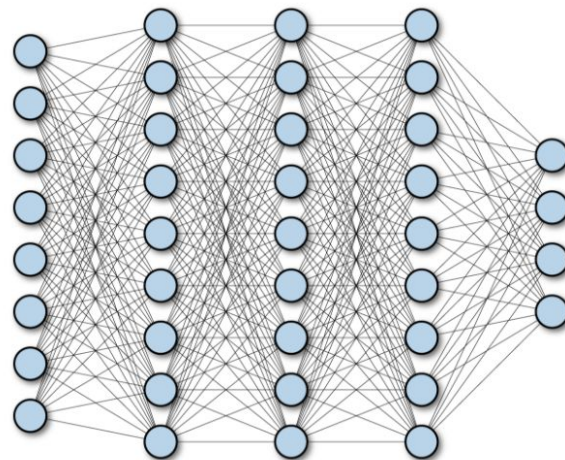
Fully connected means that each neuron in one layer is connected to every neuron in the following layer. This architecture ensures that information flows smoothly from the input to the output layer.

Connections:

- In an MLP, every neuron in one layer applies a **weighted sum** of the outputs of all neurons from the previous layer and then applies an **activation function** (such as ReLU, sigmoid, or tanh) to introduce non-linearity.

Example:

- If a hidden layer has 4 neurons, and the next hidden layer has 5 neurons, each of the 4 neurons will be connected to each of the 5 neurons in the next layer, resulting in 20 connections.



Layers of the Multilayer Perceptron: In detail

An MLP is structured into three primary types of layers, each playing a critical role in processing and transforming input data:

1. Input Layer:

- **Function:** The input layer is responsible for receiving the raw data or input features that will be processed by the network.
- **Number of Neurons:** The number of neurons in the input layer corresponds to the number of input features in the dataset. Each neuron in the input layer takes one feature from the input data.

Example:

- In an image classification problem, each pixel from the image (e.g., 28x28 pixels for MNIST) would be an input feature, resulting in 784 input neurons (one for each pixel).

In NLP tasks, the input to the MLP often comes from the **text** that needs to be classified. However, raw text cannot be fed directly into an MLP, so it first needs to be converted into numerical representations.

Common Techniques for text representation:

- **Bag of Words:** Represents a text as a vector of word counts or term frequency-inverse document frequency (TF-IDF).
- **Word Embeddings:** Converts words into dense vectors that capture semantic meaning (e.g., using pre-trained embeddings like Word2Vec or GloVe).

Example:

- Consider the sentence: "**This movie is amazing!**"
- Using **word embeddings**, each word in the sentence ("This," "movie," "is," "amazing") is transformed into a vector (e.g., 300-dimensional vector). These word vectors are concatenated or averaged to form a single vector representing the entire sentence.

Input Layer:

- The **number of neurons** in the input layer will correspond to the dimensionality of the input vector (e.g., if the sentence is represented as a 300-dimensional word embedding, there will be 300 neurons in the input layer).

2. Hidden Layer(s):

- **Function:** Hidden layers contain neurons that apply **non-linear transformations** to the inputs. These transformations enable the MLP to learn complex patterns and relationships that are not linearly separable.
- **Number of Neurons:** The number of neurons in the hidden layers can vary depending on the complexity of the problem. More neurons in a hidden layer or more hidden layers increase the capacity of the network to learn intricate patterns.
Non-Linearity: Without hidden layers, an MLP would behave like a simple linear classifier. The non-linear activation functions in the hidden layers allow the model to solve more complex tasks by introducing non-linearity into the network's decision-making process.

The hidden layers in an MLP transform the input representations (in this case, the sentence vector) to learn deeper, more complex patterns in the data.

- The neurons in the hidden layers apply **non-linear transformations** to the input, allowing the MLP to capture complex linguistic relationships, such as the interaction between words (e.g., "movie" and "amazing" indicate positive sentiment).

Example:

- The hidden layers can help the model learn that words like "amazing," "great," or "fantastic" are highly correlated with positive sentiment, even if these words weren't explicitly seen in the training data.

Number of Neurons:

- The number of neurons in the hidden layers is flexible, depending on the complexity of the task. For example, 128 or 256 neurons may be used in a hidden layer to capture the nuances in the text.

3. Output Layer:

- **Function:** The output layer provides the final result based on the transformations performed by the hidden layers. The output could be a predicted class for classification problems or a continuous value for regression problems.
- **Number of Neurons:**
 - **For classification:** If the task involves classifying objects into multiple categories (e.g., recognizing digits from 0 to 9), the output layer will have one neuron for each class.
 - **For regression:** If the task involves predicting a continuous value, there may be a single output neuron representing the predicted value.

The output layer in an MLP produces the final prediction, which, in this case, is the **sentiment** of the text.

Number of Neurons:

- For binary classification (positive or negative sentiment), there will be **one neuron** in the output layer with a **sigmoid activation function**. This neuron will output a value between 0 and 1, representing the probability of the sentence being positive or negative.

Example:

- For the sentence "**This movie is amazing!**", the output neuron might output a value close to 1, indicating **positive sentiment**.

How MLPs Overcome The Limitation

The key difference between a **single-layer perceptron** and an **MLP** is the introduction of **hidden layers** and **non-linear activation functions**. These components allow MLPs to solve **non-linearly separable problems**, like XOR, by applying complex transformations to the input data.

a. Hidden Layers and Non-Linearity

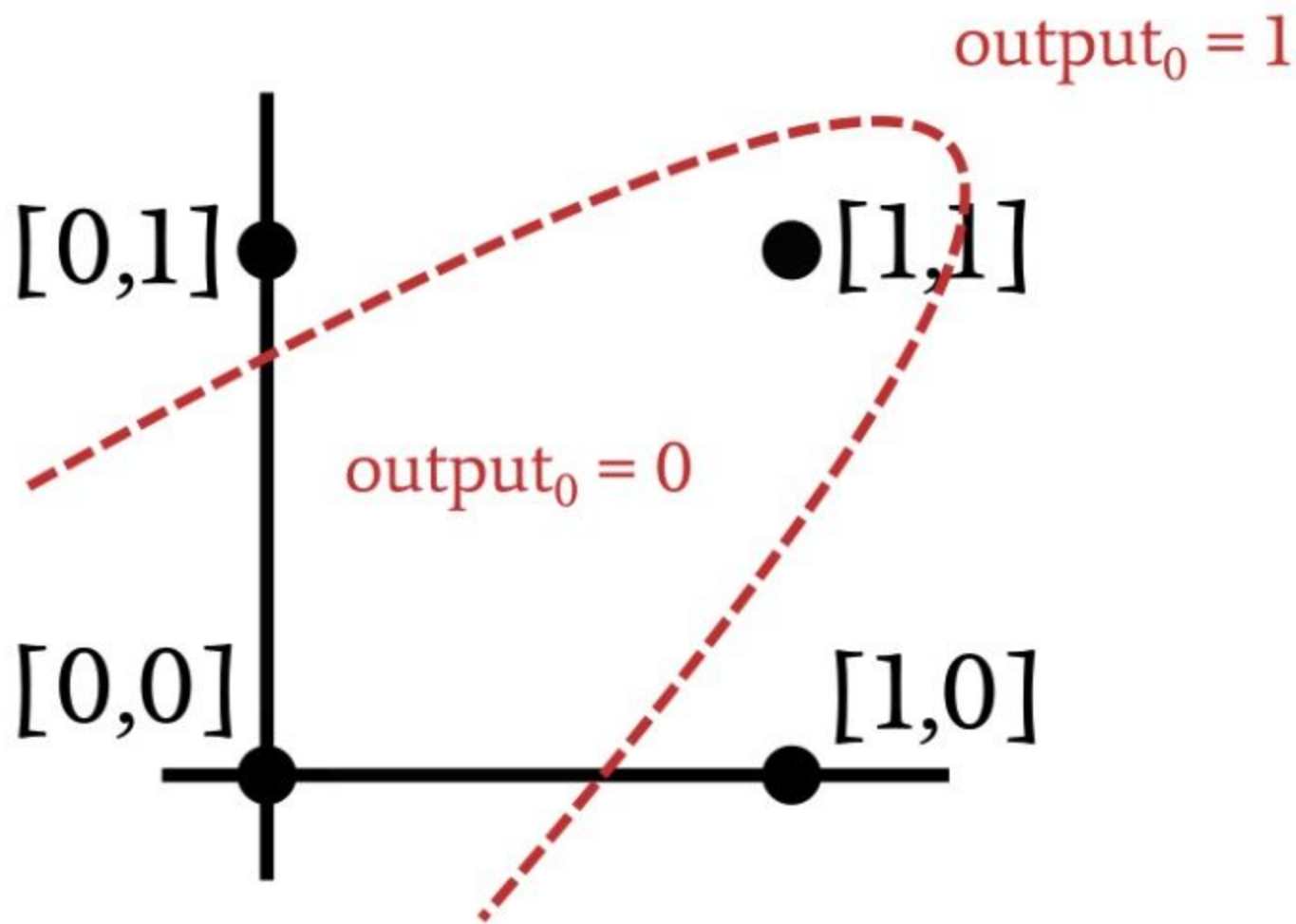
- **Hidden layers** play a crucial role in transforming input data in a way that makes it linearly separable in a new, higher-dimensional space.
- In an MLP, each hidden layer applies a **non-linear transformation** to the data, meaning it can model more complex decision boundaries than a straight line.

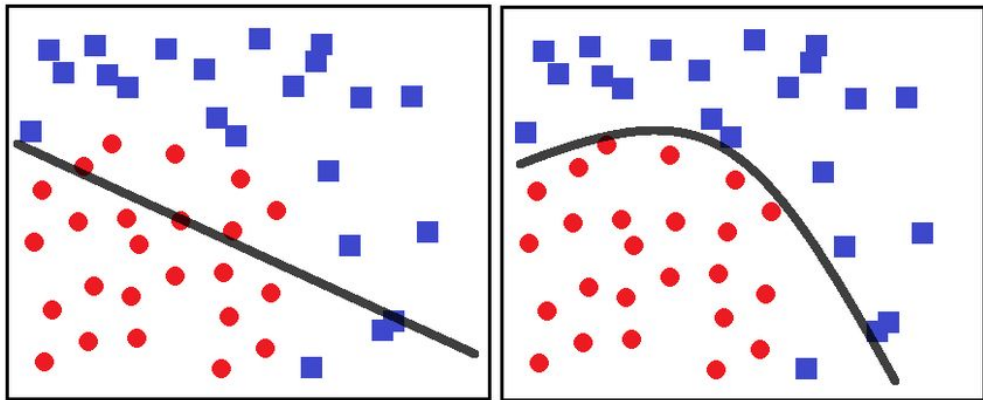
Non-Linearity in MLPs:

- The **activation functions** in the hidden layers introduce non-linearity. This allows the MLP to create **curved decision boundaries** that can separate data points that are not linearly separable in the original input space.

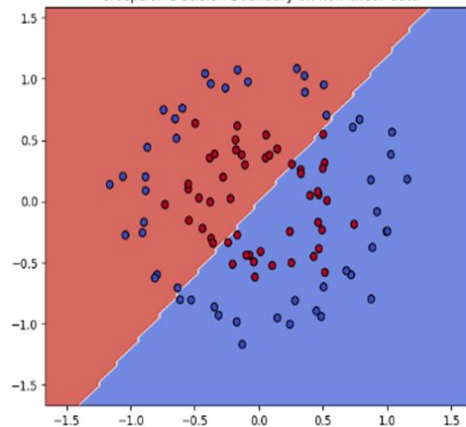
Example:

- If you're trying to classify circular data, a single-layer perceptron might fail because a straight line cannot separate the data points. An MLP, however, can create a curved boundary using non-linear transformations.

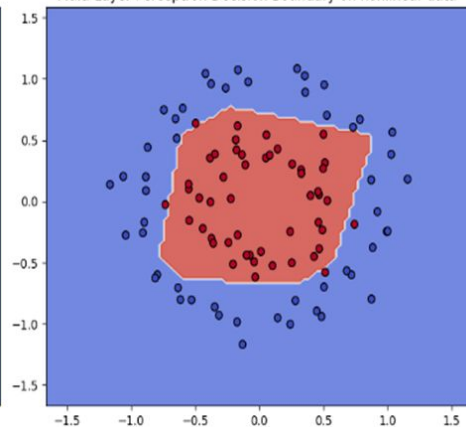




Perceptron Decision Boundary on non linear data



Multi-Layer Perceptron Decision Boundary on nonlinear data



Forward Propagation in Multilayer Perceptron

Forward propagation involves moving the inputs through the MLP, layer by layer, to produce the final output. Each layer in the network applies transformations to the data, enabling the model to learn patterns and make predictions.

1. Input Layer:

- The input layer receives the raw input data (e.g., features from an image, text, or numerical values) and passes it to the next layer (the first hidden layer).

2. Hidden Layers:

- Each hidden layer consists of neurons that compute **weighted sums** of the inputs from the previous layer and apply **activation functions** to introduce non-linearity.

3. Output Layer:

- The output layer produces the final prediction, which could be a class label in classification tasks or a numerical value in regression tasks.

Activation Functions

Activation functions are **mathematical functions** applied to the output of each neuron in a neural network. They transform the weighted sum of inputs to produce the final output of a neuron, which is then passed to the next layer or, in the case of the output layer, used as the network's prediction.

Key Points:

- The activation function introduces **non-linearity** into the neural network, which allows it to model complex patterns.
- Activation functions are applied after the neuron calculates the **weighted sum** of its inputs and adds a **bias** term.

Mathematically, if a neuron computes a weighted sum of inputs $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$, the activation function $f(z)$ transforms z into the neuron's final output:

$$y = f(z)$$

Ideal Activation Functions: Characteristics

- Non-linear
- Differentiable
 - Easy to calculate the derivative for gradient descent
- Computationally Inexpensive
 - Calculating the derivative is simple, easy and fast leading to faster training
- Zero-Centered
 - Or normalized.
 - Empirically its proven, if we have normalized values (for i/p and corresponding layers) then the NN converges faster
 - Mean of the activation function will be 0, e.g., Tanh
- Non-saturating
 - Doesn't squeeze the value in a particular range hence avoiding vanishing gradient problem
 - Saturating functions, e.g., Sigmoid, tanh
 - E.g., Relu is non-saturating

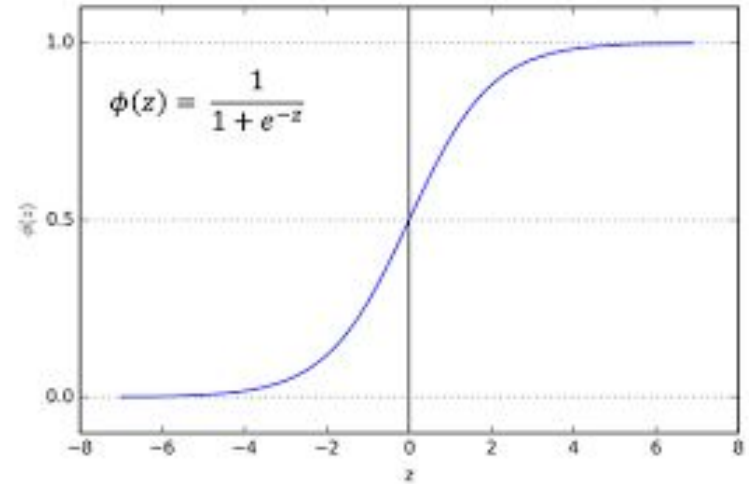
Sigmoid Activation Function

Mathematical Form:

The **sigmoid activation function** is defined as:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Where z is the weighted sum of inputs to the neuron.



Properties:

- **Range:** The output of the sigmoid function is always between 0 and 1. This makes it ideal for problems where the output represents probabilities, particularly in binary classification tasks.
- **Smooth and Differentiable:** The sigmoid function is **smooth** and **differentiable**, which is crucial for backpropagation and optimization in neural networks. The smooth nature allows for efficient computation of gradients.

Advantages:

- **Probabilistic Interpretation:** The sigmoid function is often used when the output needs to be interpreted as a probability, as the output is bounded between 0 and 1.
- **Normalization:** The sigmoid function **squashes** the input values into a small range, which can help with normalization and prevent extremely large values.

Disadvantages:

- **Vanishing Gradient Problem:** For very large or very small input values, the sigmoid function outputs values close to 0 or 1. This causes the gradient of the function to become very small (close to 0), which leads to the **vanishing gradient problem**. As a result, the network struggles to learn and adjust the weights, particularly in deep layers.
- **Saturated Neurons:** When the input z is very large or very small, the sigmoid neuron becomes **saturated**, meaning small changes in the input do not significantly affect the output, leading to poor learning.

Use Case:

- The sigmoid activation function is primarily used in **binary classification problems**, where the output is either 0 or 1. For example, in logistic regression and binary classifiers, the sigmoid function is often used in the **output layer**.